

Running Qiskit on IBM Quantum Hardware

Instructions for FYS5419/9419

Spring 2026

What you will do

- 1 Create an IBM Quantum Platform / IBM Cloud account and select an access plan
- 2 Initialize Qiskit Runtime credentials locally
- 3 Discover available backends (QPUs and simulators)
- 4 Run circuits using Qiskit Runtime primitives (Sampler / Estimator)
- 5 Monitor jobs and retrieve results

References: IBM Quantum Platform Quickstart and guides on account setup, credentials, and execution primitives.

(Quickstart, Save credentials, Execute on hardware, Get started with primitives).

Step 1: Create an account and choose a plan

- Create an account on the **IBM Quantum Platform / IBM Cloud**.
- IBM Quantum Platform offers an **Open (free)** access path and paid plans (depending on region/org).
- For QPU jobs through **Qiskit Runtime**, you typically need access to a Runtime instance (linked to an IBM Cloud account).

IBM docs: [account setup](#) and [plan overview](#).

Set up IBM Cloud account, [Plans overview](#), [Platform registration](#).

Step 2: Create / select a Qiskit Runtime service instance

- Log in to IBM Quantum Platform and ensure the correct **account** and **region** are selected.
- Verify you have a **Qiskit Runtime service instance**; if not, create one.
- You will later connect to this instance from Python.

IBM docs: [initializing the Runtime service account / instance](#).
Initialize your Qiskit Runtime service account.

Step 3: Install Qiskit and IBM Runtime packages

- Use a virtual environment (recommended) and install Qiskit + IBM Runtime tooling.

```
python3 -m venv .venv
source .venv/bin/activate

pip install --upgrade pip
pip install qiskit qiskit-ibm-runtime qiskit-ibm-provider
```

IBM quickstart emphasizes using a virtual environment.

Quickstart.

Step 4: Save credentials locally (token / API key)

- In IBM Quantum Platform, obtain your API credentials (token/API key).
- Save them in your local environment so you do not hard-code secrets in scripts.

```
from qiskit_ibm_runtime import QiskitRuntimeService

# Option A (common): save for later use (interactive)
QiskitRuntimeService.save_account(
    channel="ibm_quantum_platform",
    token="YOUR_TOKEN_HERE",
    # instance may be required depending on your plan/org:
    # instance="ibm-q/open/main" # example format; use your
    #       actual instance
    overwrite=True
)

# Later in scripts/notebooks:
service = QiskitRuntimeService(channel="ibm_quantum_platform")
```

Step 5: List available backends (QPUs and simulators)

```
from qiskit_ibm_runtime import QiskitRuntimeService

service = QiskitRuntimeService(channel="ibm_quantum_platform")

# List backends you can access
backends = service.backends()
print("Number of backends:", len(backends))
for b in backends[:10]:
    print(b.name)
```

- Choose a backend based on availability, qubit count, and queue length.
- IBM provides documentation on viewing backend information via the service API.

IBM docs: listing backends and QPU information.

Get backend information with Qiskit.

Why primitives (Sampler / Estimator)?

- IBM recommends running circuits via **Qiskit Runtime primitives**:
 - **Sampler**: returns samples / counts from circuit measurements.
 - **Estimator**: returns expectation values of observables.
- Primitives can include runtime features such as optimized execution and (optionally) error mitigation.

IBM docs: primitives and executing on hardware.

Execute on target hardware, Primitives examples, Introduction to primitives.

Example A (Sampler): run a Bell circuit on hardware

```
from qiskit import QuantumCircuit, transpile
from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2 as
    Sampler

service = QiskitRuntimeService(channel="ibm_quantum_platform")

# Pick a backend (replace with one you have access to)
backend = service.backend("ibm_brisbane") # example name

# Build circuit
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.measure([0, 1], [0, 1])

# Transpile to the backend's ISA
isa_circuit = transpile(qc, backend=backend, optimization_level
    =1)
```

Example B (Estimator): expectation value of an observable

- Estimator returns $\langle O \rangle$ for an observable O on the circuit output state.
- Typical use: VQE, QML, metrology objectives, etc.

```
from qiskit import QuantumCircuit, transpile
from qiskit.quantum_info import SparsePauliOp
from qiskit_ibm_runtime import QiskitRuntimeService, EstimatorV2
    as Estimator

service = QiskitRuntimeService(channel="ibm_quantum_platform")
backend = service.backend("ibm_brisbane") # example

# Prepare |Phi+> and estimate <ZZ>
qc = QuantumCircuit(2)
qc.h(0); qc.cx(0, 1)

obs = SparsePauliOp.from_list([("ZZ", 1.0)])

isa_circuit = transpile(qc, backend=backend, optimization_level
    =1)
```

Sessions (recommended for multiple runs)

- If you execute *many* primitive calls, using a session can reduce overhead.
- A session keeps context on the backend for a window of time (useful for iterative workflows).

```
from qiskit_ibm_runtime import QiskitRuntimeService, Session
from qiskit_ibm_runtime import SamplerV2 as Sampler

service = QiskitRuntimeService(channel="ibm_quantum_platform")
backend = service.backend("ibm_brisbane") # example

with Session(service=service, backend=backend) as session:
    sampler = Sampler(session=session)
    job = sampler.run([isa_circuit], shots=2000)
    print(job.result())
```

IBM docs: primitives + execution flow; sessions are part of Runtime usage patterns.

Execute on hardware.

Monitoring jobs and handling queues

```
# For any job returned by Sampler/Estimator:  
print("Status:", job.status())  
  
# Blocking wait (simple):  
result = job.result()  
  
# Some jobs expose logs/metadata depending on runtime  
  configuration:  
# print(job.metrics()) # if available in your environment
```

- Hardware runs are queued; queue times vary by backend load and access plan.
- Start with simulators, then validate on hardware.
- Keep shot counts modest during development; increase shots for final statistics.

IBM docs: [backend info](#) and [execution guidance](#).

Get QPU information, [Execute on hardware](#).

Practical tips for running on IBM QPUs

- **Transpile matters:** always transpile to the backend target/ISA.
- **Noise:** expect deviations from ideal counts/expectations; compare with a simulator baseline.
- **Measurement error mitigation:** primitives may support resilience options depending on plan.
- **Reproducibility:** record backend name, calibration date/time, transpiler settings, and shots.
- **Iteration:** develop locally (simulator), then run a small calibration set on hardware.

IBM docs: primitives can provide sophisticated implementations including mitigation.
Introduction to primitives.

Checklist (quick workflow)

- 1 Create IBM Quantum Platform / IBM Cloud account and select an access plan.
- 2 Create/select a Qiskit Runtime instance for your account/region.
- 3 Install packages: `qiskit`, `qiskit-ibm-runtime`, `qiskit-ibm-provider`.
- 4 Save credentials with `QiskitRuntimeService.save_account(...)`.
- 5 List backends: `service.backends()`.
- 6 Build circuit, transpile to backend, run via `Sampler` or `Estimator`.
- 7 Retrieve results and document settings.

IBM docs hub: Quickstart.